Chapter 18
Recursion

Java How to Program, 11/e

Questions? E-mail paul.deitel@deitel.com

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

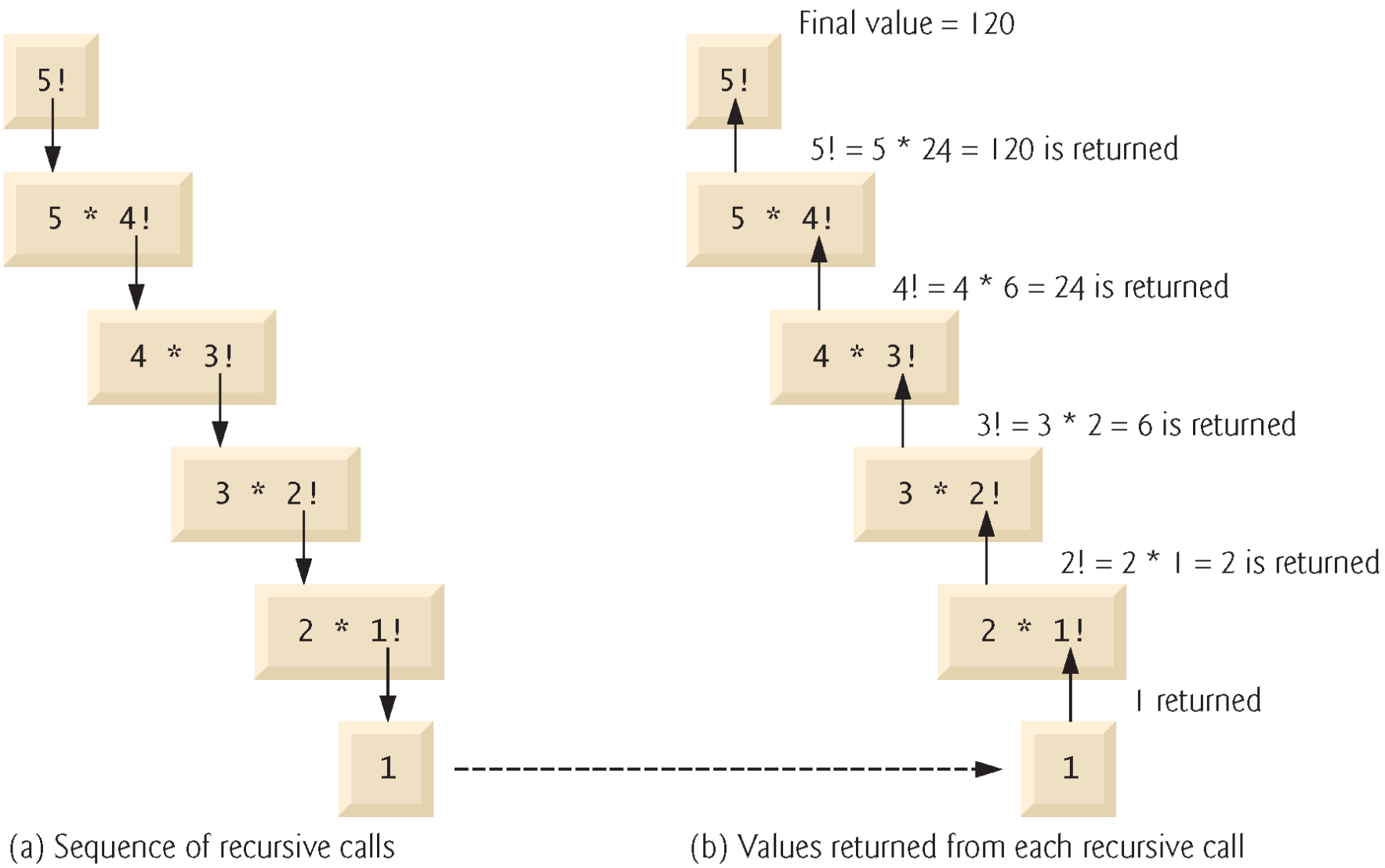


Fig. 18.2 | Recursive evaluation of 5!.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Common Programming Error 18.1

Either omitting the base case or writing the recursion step incorrectly so that it does not converge on the base case can cause a logic error known as **infinite recursion**, where recursive calls are continuously made until memory is exhausted or the method-call stack overflows. This error is analogous to the problem of an infinite loop in an iterative (nonrecursive) solution.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```java
// Fig. 18.3: FactorialCalculator.java
// Recursive factorial method.

public class FactorialCalculator {
   // recursive method factorial (assumes its parameter is >= 0)
   public static long factorial(long number) {
      if (number <= 1) { // test for base case
         return 1; // base cases: 0! = 1 and 1! = 1
      }
      else { // recursion step
         return number * factorial(number - 1);
      }
   }
```

Fig. 18.3 | Recursive factorial method. (Part 1 of 2.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```java

    public static void main(String[] args) {
        // calculate the factorials of 0 through 21
        for (int counter = 0; counter <= 21; counter++) {
            System.out.printf("%d! = %d%n", counter, factorial(counter));
        }
    }
}
```

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
...
12! = 479001600 —— 12! causes overflow for int variables
...
20! = 2432902008176640000
21! = -4249290049419214848 —— 21! causes overflow for long variables
```

Fig. 18.3 | Recursive factorial method. (Part 2 of 2.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```java
// Fig. 18.4: FactorialCalculator.java
// Recursive factorial method.
import java.math.BigInteger;

public class FactorialCalculator {
   // recursive method factorial (assumes its parameter is >= 0)
   public static BigInteger factorial(BigInteger number) {
      if (number.compareTo(BigInteger.ONE) <= 0) { // test base case
         return BigInteger.ONE; // base cases: 0! = 1 and 1! = 1
      }
      else { // recursion step
         return number.multiply(
            factorial(number.subtract(BigInteger.ONE)));
      }
   }
```

Fig. 18.4 | Factorial calculations with a recursive method. (Part 1 of 2.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```java

    public static void main(String[] args) {
       // calculate the factorials of 0 through 50
       for (int counter = 0; counter <= 50; counter++) {
          System.out.printf("%d! = %d%n", counter,
             factorial(BigInteger.valueOf(counter)));
       }
    }
 }
```

```
0! = 1
1! = 1
2! = 2
3! = 6
...
21! = 51090942171709440000 —— 21! and larger values no longer cause overflow
22! = 1124000727777607680000
...
47! = 258623241511168180642964355153611979969197632389120000000000
48! = 12413915592536072670862289047373375038521486354677760000000000
49! = 608281864034267560872252163321295376887552831379210240000000000
50! = 30414093201713378043612608166064768844377641568960512000000000000
```

Fig. 18.4 | Factorial calculations with a recursive method. (Part 2 of 2.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```java
// Fig. 18.5: FibonacciCalculator.java
// Recursive fibonacci method.
import java.math.BigInteger;

public class FibonacciCalculator {
   private static BigInteger TWO = BigInteger.valueOf(2);

   // recursive declaration of method fibonacci
   public static BigInteger fibonacci(BigInteger number) {
      if (number.equals(BigInteger.ZERO) ||
            number.equals(BigInteger.ONE)) { // base cases
         return number;
      }
      else { // recursion step
         return fibonacci(number.subtract(BigInteger.ONE)).add(
            fibonacci(number.subtract(TWO)));
      }
   }
```

Fig. 18.5 | Recursive fibonacci method. (Part 1 of 3.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```java

    public static void main(String[] args) {
        // displays the fibonacci values from 0-40
        for (int counter = 0; counter <= 40; counter++) {
            System.out.printf("Fibonacci of %d is: %d%n", counter,
              fibonacci(BigInteger.valueOf(counter)));
        }
    }
}
```

Fig. 18.5 | Recursive fibonacci method. (Part 2 of 3.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
Fibonacci of 0 is: 0
Fibonacci of 1 is: 1
Fibonacci of 2 is: 1
Fibonacci of 3 is: 2
Fibonacci of 4 is: 3
Fibonacci of 5 is: 5
Fibonacci of 6 is: 8
Fibonacci of 7 is: 13
Fibonacci of 8 is: 21
Fibonacci of 9 is: 34
Fibonacci of 10 is: 55
...
Fibonacci of 37 is: 24157817
Fibonacci of 38 is: 39088169
Fibonacci of 39 is: 63245986
Fibonacci of 40 is: 102334155
```

Fig. 18.5 | Recursive fibonacci method. (Part 3 of 3.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

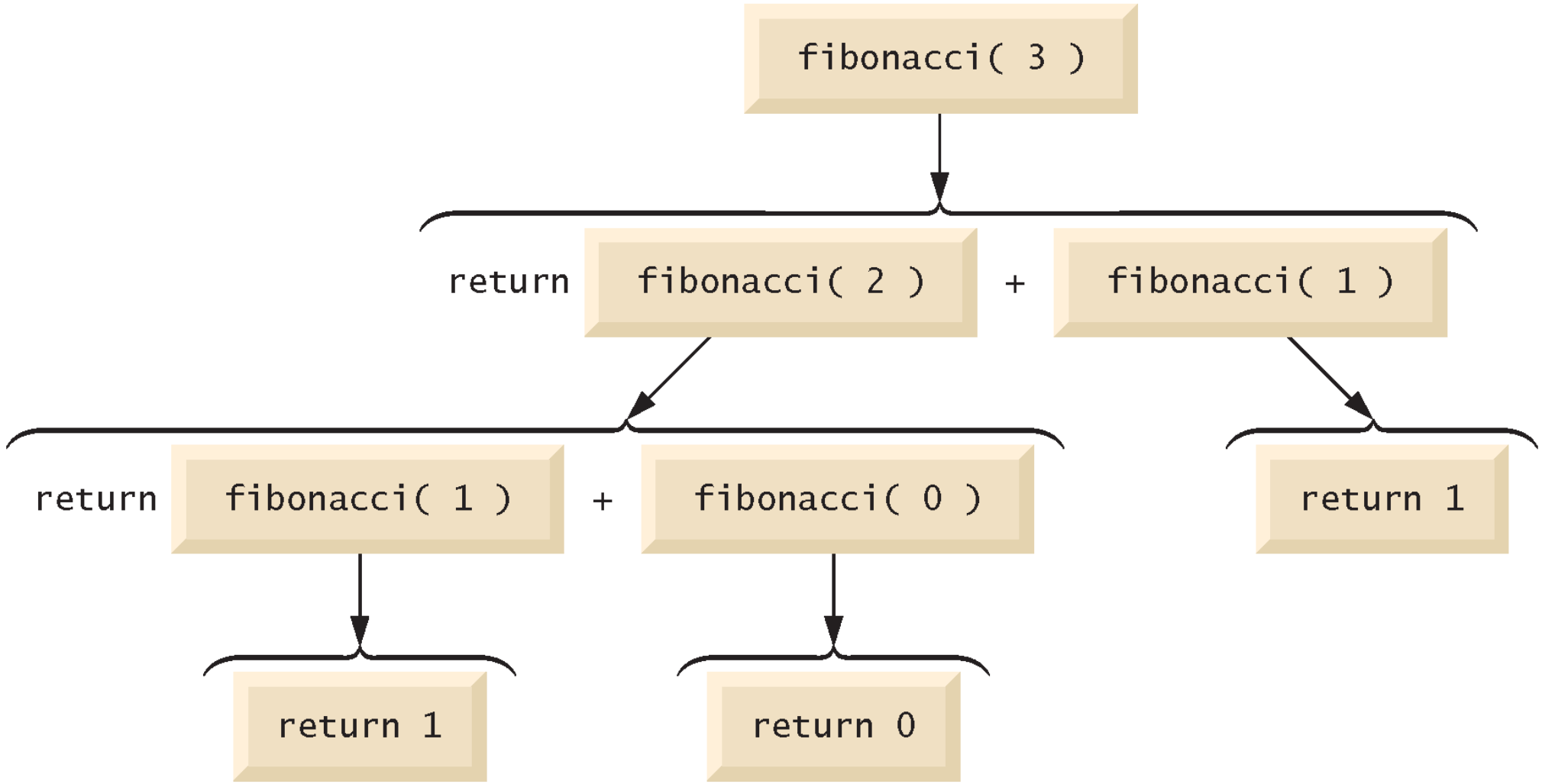
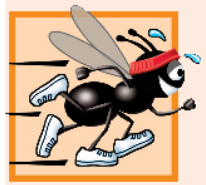


Fig. 18.6 | Set of recursive calls for `fibonacci(3)`.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.



Performance Tip 18.1

Avoid Fibonacci-style recursive programs, because they result in an exponential "explosion" of method calls.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

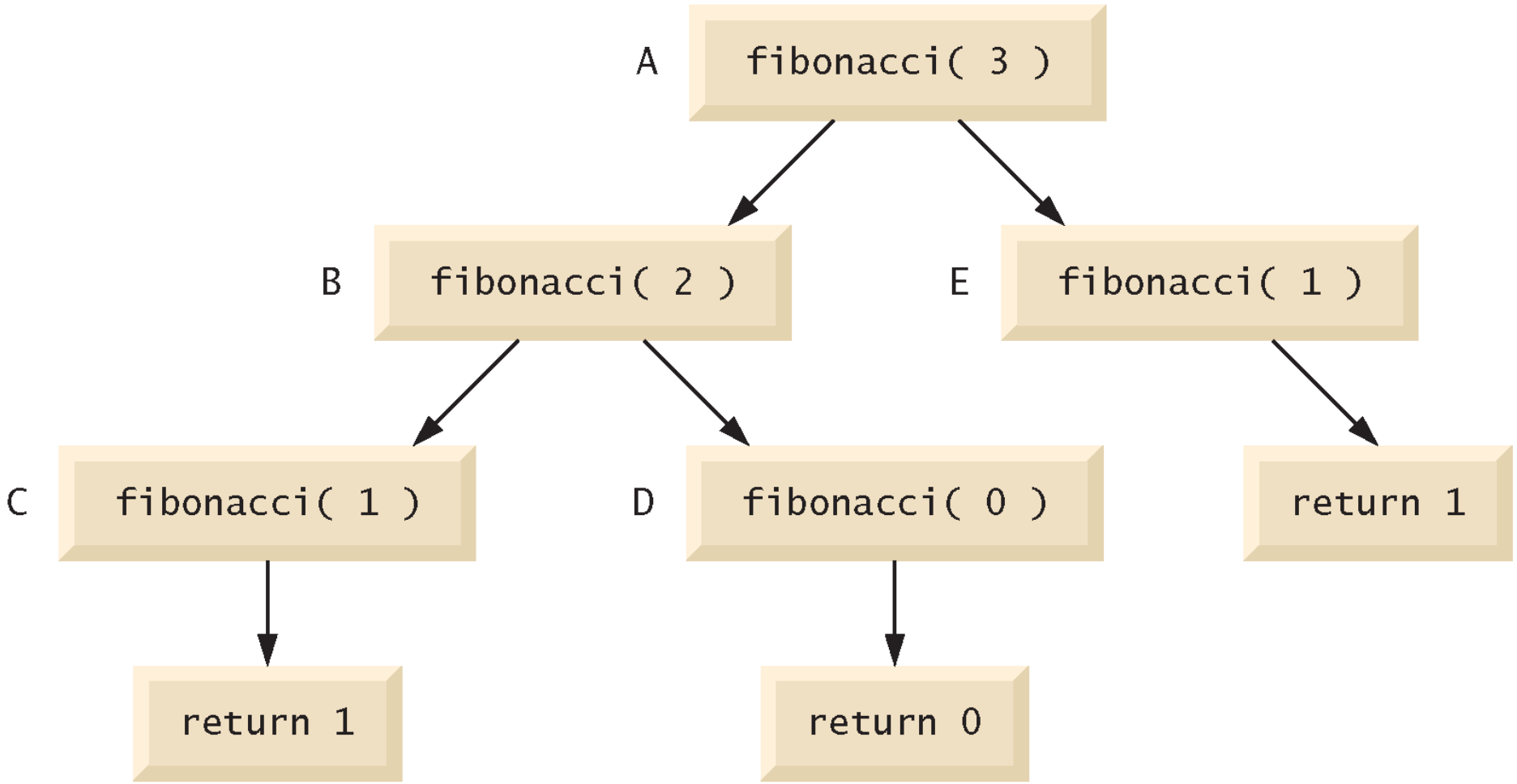


Fig. 18.7 | Method calls made within the call `fibonacci(3)`.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

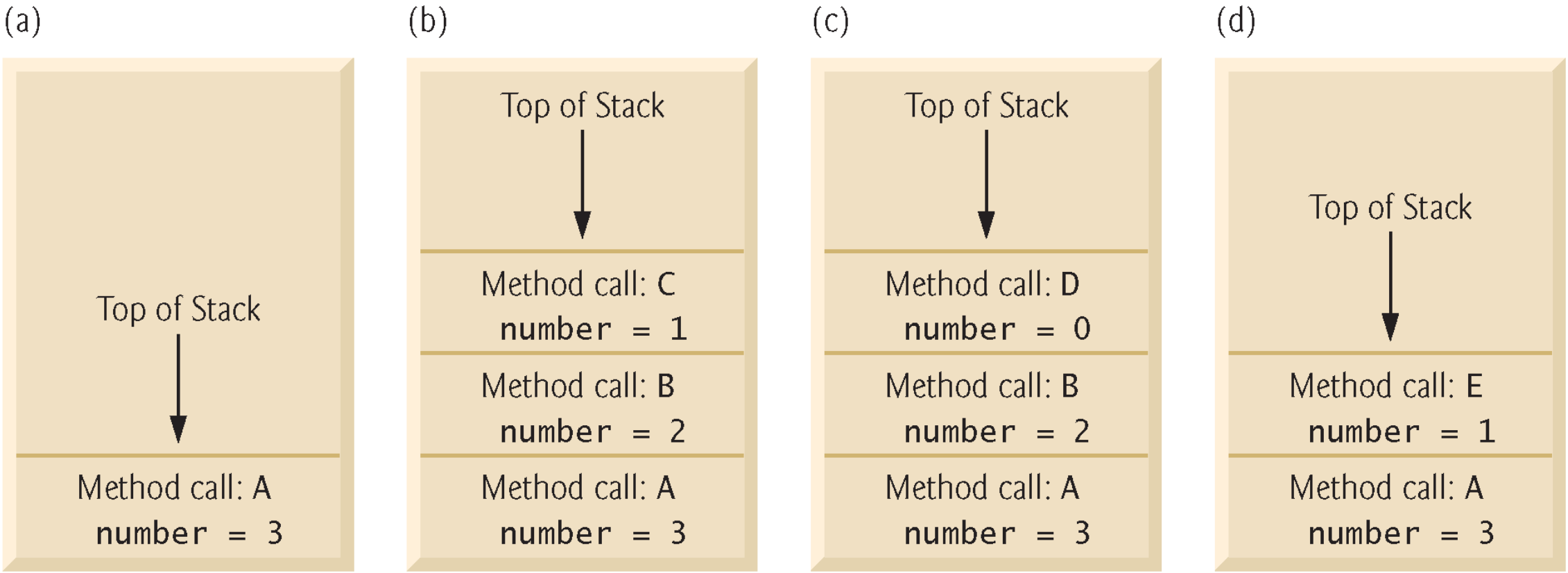


Fig. 18.8 | Method calls on the program-execution stack.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```java
// Fig. 18.9: FactorialCalculator.java
// Iterative factorial method.

public class FactorialCalculator {
   // iterative declaration of method factorial
   public long factorial(long number) {
      long result = 1;

      // iteratively calculate factorial
      for (long i = number; i >= 1; i--) {
         result *= i;
      }

      return result;
   }

```

Fig. 18.9 | Iterative factorial method. (Part 1 of 2.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```java
    public static void main(String[] args) {
        // calculate the factorials of 0 through 10
        for (int counter = 0; counter <= 10; counter++) {
            System.out.printf("%d! = %d%n", counter, factorial(counter));
        }
    }
}
```

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
```

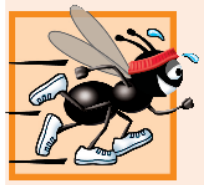
Fig. 18.9 | Iterative factorial method. (Part 2 of 2.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Software Engineering Observation 18.1

Any problem that can be solved recursively can be solved iteratively and vice versa. A recursive approach is preferred over an iterative approach when the recursive approach more naturally mirrors the problem and results in a program that's easier to understand and debug. A recursive approach can often be implemented with fewer lines of code. Another reason to choose a recursive approach is that an iterative one might not be apparent.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.



Performance Tip 18.2

When performance is crucial, you might want to try various iterative and recursive approaches to see which achieve your goal.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Common Programming Error 18.2

Accidentally having a nonrecursive method call itself either directly or indirectly through another method can cause infinite recursion.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

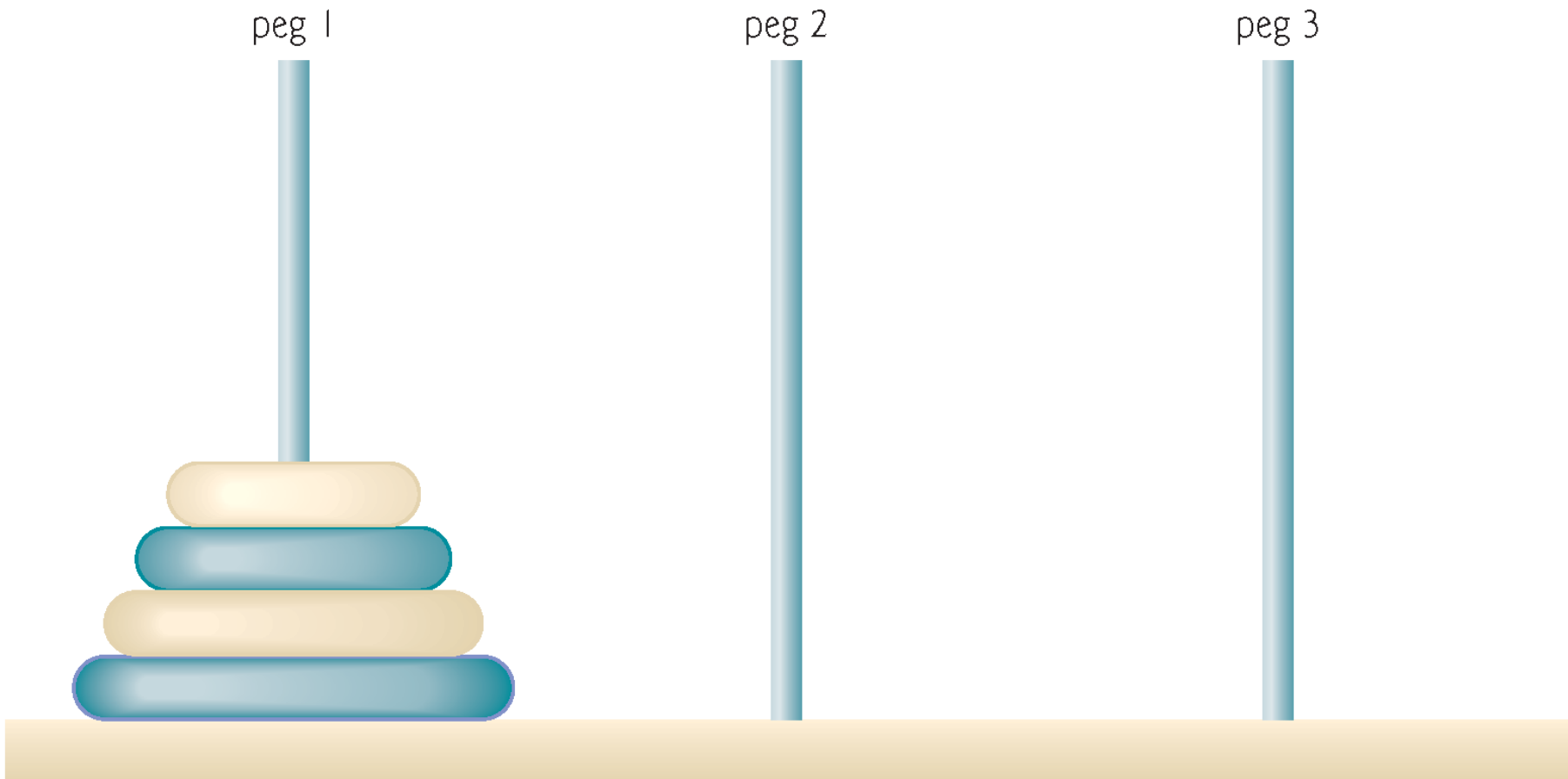


Fig. 18.10 | Towers of Hanoi for the case with four disks.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```java
// Fig. 18.11: TowersOfHanoi.java
// Towers of Hanoi solution with a recursive method.
public class TowersOfHanoi {
   // recursively move disks between towers
   public static void solveTowers(int disks, int sourcePeg,
      int destinationPeg, int tempPeg) {
      // base case -- only one disk to move
      if (disks == 1) {
         System.out.printf("%n%d --> %d", sourcePeg, destinationPeg);
         return;
      }

      // recursion step -- move (disk - 1) disks from sourcePeg
      // to tempPeg using destinationPeg
      solveTowers(disks - 1, sourcePeg, tempPeg, destinationPeg);

      // move last disk from sourcePeg to destinationPeg
      System.out.printf("%n%d --> %d", sourcePeg, destinationPeg);

      // move (disks - 1) disks from tempPeg to destinationPeg
      solveTowers(disks - 1, tempPeg, destinationPeg, sourcePeg);
   }
```

Fig. 18.11 | Towers of Hanoi solution with a recursive method. (Part 1 of 2.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.